

EC8208 – CA03 Prototype Demo & Presentation Cheat-Sheet

MediConnect – Microservices Clinic Appointment Booking Platform (running prototype)

30-Second Pitch

A **running microservices prototype** of MediConnect. A **React SPA** calls an **API Gateway** that routes to **4 independent Node.js services** (Auth, Appointment, Records, Notification). Each owns its **own MongoDB database**. Services talk **synchronously via REST** and **asynchronously via RabbitMQ**. Everything runs with **one command** using **Docker Compose**.

Run It (deployment demo)

```
cd CA03_Prototype
```

```
docker compose up --build
```

Web app localhost:3000 • **Gateway** :8080

RabbitMQ UI :15672 (guest/guest) – show events live!

Demo Script (say this while clicking)

1. Register + login as **patient** (Auth issues a JWT).
2. **Find a Doctor** → Book a slot (Appointment reserves slot, simulates payment, **publishes** `appointment.booked`).
3. Show **My Appointments + Notifications** (created *async* from the event).
4. Login as **doctor** → **Consultation Records** (record auto-created from same event) → add notes + prescription → Complete.
5. Back as patient → **My Records** → prescription now shows.
6. Open **RabbitMQ UI** → show the two queues that consumed the event.

Communication (know this cold)

- **Sync (REST)**: Browser → Gateway → service. Plus Records → Appointment (validate appointment).
- **Async (events)**: Appointment **publishes** `appointment.booked`; Notification + Records **consume** it independently (fan-out via RabbitMQ topic exchange `mediconnect.events`).

Services (each = own DB, layered)

Service	Port	Does / key endpoints
Auth	4001	register, login, JWT <code>/api/auth/*</code>
Appointment	4002	doctor search, booking, publishes event
Records	4003	consult notes + e-prescription <code>/api/records</code>
Notification	4004	booking confirmations <code>/api/notifications</code>
API Gateway	8080	JWT, routing, rate limit
MongoDB	27017	one DB per service
RabbitMQ	5672	event bus (+UI 15672)

Layered Code (per service)

`routes.js` (controller) → `controller.js` (business logic) → `model.js` (data access) → `events.js` (publish/consume).

Security in the Prototype

JWT issued by Auth (jsonwebtoken); **Gateway verifies** every protected route, forwards identity as `x-user-id/role` headers; **bcrypt** password hashing; **role-based** checks (patient/doctor); gateway **rate limiting**.

CA03 Rubric Coverage

UI (React) • Business logic (`controller.js`) • Data layer (MongoDB models) • 3+ functions (auth, book, records) • error handling (validation + central middleware) • security (JWT/bcrypt/RBAC) • deployment (Docker Compose).

Honest Simplifications (own them)

All-Node (no Spring Boot) • MongoDB only (design also had PostgreSQL) • Docker Compose (design: Kubernetes) • payment simulated • doctor view shows all records.

Talking Points – Key Implementation Decisions (Choice → Why → Trade-off)

Decision	Why (in the prototype)	Trade-off / honest note
API Gateway only public	Single entry; JWT + routing + rate limit; services hidden on the Docker network	Gateway is a SPOF – would replicate it in production
DB-per-service (one MongoDB, separate DB names)	Demonstrates loose coupling; each service owns <code>auth_db</code> , <code>appointment_db</code> , etc.	Logical (not physical) separation to keep the demo light
RabbitMQ events (fan-out)	Booking stays fast; notification + record creation happen asynchronously; resilient if a consumer is down	Eventual consistency; a broker to run
Docker Compose	One-command deploy of 7 containers with health checks + retry logic	Not full Kubernetes/HPA (that is the CA01 production design)
nginx proxies /api to gateway	Frontend calls relative <code>/api</code> ; same-origin, so no CORS issues	Extra nginx container in front of the SPA
Gateway path rewrite (function-based)	Correctly maps <code>/api/doctors/123</code> to <code>/doctors/123</code> after Express strips the mount path	(A real bug we found and fixed – good “hardest problem” story)